What is a Closure?

A **closure** is a function that **remembers and has access to variables from its outer scope** — even after the outer function has finished executing.

> If you know JavaScript, closures work **exactly the same way** in Go

---

File Setup

```
golang-course/
└── 14-closures/
    └── closures.go
```

```go
package main

import "fmt"

func main() {
    // code goes here
}
```

Understanding the Problem First

Normally when a function finishes executing:

- It is removed from the **call stack**
- All its **variables are deleted** from memory
- Those variables are **gone forever**

```go
func counter() {
    count := 0
    count++
    fmt.Println(count)
}

counter()  // 1
counter()  // 1 — resets every time! count is gone after each call
counter()  // 1 — still 1, not remembering previous value
```

This is normal behavior. But what if you want a function to **remember** its previous state? That's where closures come in.

Building a Counter with Closures

```
func counter() func() int {
    count := 0

    return func() int {
        count += 1
        return count
    }
}
```

Breaking it down step by step:

Step 1 — Outer function:

```
func counter() func() int {
```

- `counter` is the outer function
- It receives nothing
- It returns a **function** — `func() int` (a function that takes nothing and returns int)

Step 2 — The variable in outer scope:

```
count := 0
```

- `count` lives inside `counter()`
- Normally would be deleted when `counter()` finishes
- But because of closure — it **stays alive**

Step 3 — The inner anonymous function:

```
return func() int {
    count += 1
    return count
}
```

- This is an **anonymous function** being returned
- It accesses `count` from the **outer scope**

- This creates a closure — the inner function **closes over** `count`
- `count` will now live as long as this inner function lives

Calling the Closure

```
func main() {
    increment := counter()  // counter() returns the inner function

    fmt.Println(increment())  // Output: 1
    fmt.Println(increment())  // Output: 2
    fmt.Println(increment())  // Output: 3
}
```

What's happening step by step:

1. `counter()` is called → creates `count = 0` → returns inner function
2. Inner function is stored in `increment`
3. `increment()` is called → `count` becomes 1 → returns 1
4. `increment()` is called again → `count` becomes 2 → returns 2
5. `increment()` is called again → `count` becomes 3 → returns 3

`count` is remembering its value between calls! This is the magic of closures.

Why Does This Work? — The Call Stack Explanation

Normally:

```
counter() called
→ count = 0 created in memory
→ function finishes
→ count DELETED from memory ← normal behavior
```

With Closure:

```
counter() called
→ count = 0 created in memory
→ inner function is returned
```

→ inner function has a REFERENCE to count
→ because inner function still holds reference → count is NOT deleted
→ count stays alive in memory
→ every time increment() is called → count is updated and remembered

The key insight:

> When an inner function **references a variable from its outer scope**, that variable is **captured** by the closure and kept alive in memory as long as the inner function exists

---

Visualizing Closure Memory

```
MEMORY:

┌──────────────────────────────┐
│  closure environment         │
│  ┌──────────┐                │
│  │ count = 0 │ ← captured     │
│  └──────────┘                │
│                              │
│  increment = func() {        │
│      count += 1  ← uses it   │
│      return count            │
│  }                           │
└──────────────────────────────┘
```

Each call to increment() → modifies the SAME count in memory

Multiple Independent Closures

Each call to `counter()` creates a **completely separate** closure with its own `count`:

```
func main() {
    increment1 := counter()  // own count variable
    increment2 := counter()  // different count variable

    fmt.Println(increment1())  // 1
    fmt.Println(increment1())  // 2
    fmt.Println(increment1())  // 3

    fmt.Println(increment2())  // 1 — completely independent!
    fmt.Println(increment2())  // 2
}
```

- `increment1` and `increment2` each have their **own separate `count`**
- They don't share state
- Each closure is independent

What "Closing Over" Means

The term **closure** comes from the idea that the inner function **closes over** (captures/encloses) variables from its surrounding scope:

```
func counter() func() int {
    count := 0          // ← this variable is "closed over"

    return func() int {
        count += 1      // ← inner function closes over 'count'
        return count
    }
}
```

The inner function doesn't just read `count` at one moment — it maintains a **live reference** to it. Any changes made inside the inner function **persist**.

Practical Uses of Closures

1. Counters and ID generators:

```
func idGenerator() func() int {
    id := 0
    return func() int {
        id++
        return id
    }
}

nextID := idGenerator()
fmt.Println(nextID())  // 1
fmt.Println(nextID())  // 2
fmt.Println(nextID())  // 3
```

2. Memoization (caching results):

```
func memoize() func(int) int {
    cache := map[int]int{}

    return func(n int) int {
        if val, ok := cache[n]; ok {
            return val  // return cached result
        }
        result := n * n
        cache[n] = result  // store in cache
        return result
    }
}
```

3. Middleware and configuration:

```
func multiplier(factor int) func(int) int {
    return func(n int) int {
        return n * factor  // 'factor' is closed over
    }
}

double := multiplier(2)
triple := multiplier(3)
```

```
fmt.Println(double(5))  // 10
fmt.Println(triple(5))  // 15
```

Full Code Example

```
package main

import "fmt"

// Closure — counter example
func counter() func() int {
    count := 0

    return func() int {
        count += 1
        return count
    }
}

func main() {

    // Basic closure usage
    increment := counter()

    fmt.Println(increment())  // 1
    fmt.Println(increment())  // 2
    fmt.Println(increment())  // 3

    // Independent closures
    increment1 := counter()
    increment2 := counter()

    fmt.Println(increment1())  // 1
    fmt.Println(increment1())  // 2

    fmt.Println(increment2())  // 1 — independent!
    fmt.Println(increment2())  // 2
}
```

Running the Program

go run 14-closures/closures.go

Closures vs Normal Functions

Feature	Normal Function	Closure
Remembers state	❌ No — resets every call	✅ Yes — state persists
Access outer variables	❌ No	✅ Yes — closes over them
Each instance independent	—	✅ Yes — own environment
Variables deleted after call	✅ Yes	❌ No — kept alive

Go vs JavaScript Closures

If you know JavaScript:

```
// JavaScript
function counter() {
    let count = 0
    return function() {
        count++
        return count
    }
}

// Go — almost identical!
func counter() func() int {
    count := 0
    return func() int {
        count++
        return count
    }
}
```

The concept is **exactly the same** — only syntax differs slightly.

Summary Table

Concept	Explanation
Closure	Function that captures variables from outer scope
Closing over	Inner function keeping reference to outer variable
State persistence	Captured variables survive outer function completion
Independent closures	Each call to outer function creates separate environment
Call stack	Normally variables deleted — closure prevents this

---

Key Takeaways

1. A **closure** is an inner function that captures variables from its outer function's scope
2. Normally variables are deleted when a function finishes — closures **prevent this**
3. The inner function maintains a **live reference** to the outer variable — not just a copy
4. Every time the closure is called, it **modifies and remembers** the same variable
5. Each call to the outer function creates a **completely independent** closure
6. Multiple closures from the same outer function do **not share state**
7. Closures work the same way in Go as in **JavaScript**
8. Common uses — counters, ID generators, caching, configuration
9. The `count += 1` inside the inner function updates the **same `count`** in memory every time
10. Closures are possible because Go is a **garbage collected** language — it keeps variables alive as long as something references them

Great question! This confusion is very common. Let me clear it up completely.

Your Doubt

You're thinking:

> "Each time I call `increment()`, the inner function runs and finishes — so shouldn't `count` get deleted too?"

The answer is **NO** — and here's exactly why:

The Key Misunderstanding

There are **TWO different things** happening here:

```
increment := counter()    // LINE 1 — outer function called ONCE

fmt.Println(increment())  // LINE 2 — inner function called
fmt.Println(increment())  // LINE 3 — inner function called
fmt.Println(increment())  // LINE 4 — inner function called
```

Line	What happens
LINE 1	`counter()` runs ONCE → creates `count = 0` → returns inner function → `counter()` is done
LINE 2,3,4	You are calling the **same inner function** that was already returned and stored in `increment`

The Inner Function Never Gets "Deleted"

When you do:

```
increment := counter()
```

The inner function is **returned and stored** in `increment`. It now lives inside `increment` — it is NOT deleted.

Think of it like this:

```
counter() runs → produces a function → hands it to 'increment'
                                  ↓
                        'increment' now HOLDS
```

that function permanently
(until the program ends or
increment goes out of scope)

So when you call `increment()` three times — you are calling the **same function** that is sitting inside `increment`. It is NOT being deleted between calls. It is just being **paused and resumed**.

What Actually Gets Deleted vs What Stays

```
func counter() func() int {   // ← THIS function runs once and finishes
    count := 0                // ← count is created here

    return func() int {       // ← THIS function is returned and STORED
        count += 1            //   in 'increment' — never deleted
        return count
    }
}
```

Thing	When it finishes/deleted
`counter()` outer function	Finishes after LINE 1 — done forever
`count` variable	❌ NOT deleted — because inner function still references it
Inner function	❌ NOT deleted — lives inside `increment` variable

The Rule

> A variable is only deleted from memory when **nothing references it anymore**

Since the inner function (stored in `increment`) still has a reference to `count` — Go's garbage collector says:

> "Something is still using `count` — I cannot delete it"

So `count` stays alive.

Simple Real Life Analogy

Think of `increment` like a **TV remote** and `count` like the **TV's channel number**:

- `counter()` is the **factory** that built the remote — factory's job is done after making it
- `increment` is the **remote** sitting in your hand
- `count` is the **channel number stored in the TV**
- Every time you press the button (`increment()`) → channel number goes up
- The remote is still in your hand between button presses — it's not thrown away
- The channel number is remembered in the TV — not reset

Proof with Code

```
increment := counter()

// 'increment' is holding the inner function — NOT deleted
// 'count' is alive because 'increment' references it

fmt.Println(increment())  // inner function runs, pauses → count = 1
// inner function is PAUSED, not deleted
// count is still 1 in memory

fmt.Println(increment())  // inner function resumes → count = 2
// paused again, count still 2

fmt.Println(increment())  // resumes again → count = 3
```

When WILL count get deleted?

`count` will only be deleted when `increment` itself goes out of scope:

```
func main() {
    increment := counter()
    fmt.Println(increment())  // 1
    fmt.Println(increment())  // 2
}
// HERE — main() ends → increment goes out of scope
```

// → nothing references count anymore
// → NOW Go's garbage collector deletes count

One Line Summary

> `increment()` calling the inner function does NOT delete it — the inner function is **stored inside `increment` variable** and stays alive between calls, and so does `count` because the inner function still references it.

YES! You got it! That's exactly the reason!

Let me confirm your understanding:

You Are 100% Correct

```
func main() {
   increment := counter()    // inner function stored HERE

   fmt.Println(increment())  // call 1
   fmt.Println(increment())  // call 2
   fmt.Println(increment())  // call 3
}
```

`increment` is a **variable sitting in `main()`** — and `main()` is running the entire time.

So:

- `increment` is alive → because `main()` is still running
- `count` is alive → because `increment` still references it
- Nothing gets reset → because nothing gets deleted

The Reset Confusion — Cleared

You were thinking of this:

```
// This WOULD reset every time
func counter() {
    count := 0    // created fresh every call
    count++
    fmt.Println(count)
}

counter()  // count created → 1 → deleted
counter()  // count created → 1 → deleted
counter()  // count created → 1 → deleted
```

Here `counter()` is called 3 times → `count` is **created and deleted** each time → always resets to 1.

But in your closure code:

```
increment := counter()  // counter() called ONLY ONCE
                        // count created ONLY ONCE
                        // inner function stored in increment

increment()  // same count → becomes 1
increment()  // same count → becomes 2
increment()  // same count → becomes 3
```

`counter()` is called **only ONE time** — so `count` is created **only ONE time**.

The Golden Rule

Situation	count behaviour
`counter()` called multiple times	Resets — new `count` each time
`increment()` called multiple times	Does NOT reset — same `count`

These are **two completely different functions** being called!

Super Simple Way to Remember

counter() = the FACTORY → runs once → creates count → shuts down
increment() = the PRODUCT → runs many times → uses same count

The factory ran once. The product keeps working. The factory shutting down doesn't destroy what it made — because `main()` is holding onto it through `increment`.

Your Exact Reasoning Was Right

> **"is it because increment is in main(), so it long lasts?"**

YES — exactly this.

- `main()` runs from start to end of program
- `increment` lives inside `main()`
- So `increment` stays alive the whole time
- `increment` holds the inner function
- Inner function holds reference to `count`
- So `count` also stays alive the whole time
- Nothing resets — same `count` all 3 times

You understood it perfectly! 🎯

Short Answer — NO, not entire execution. YES, till it is referenced.

The Rule is Simple:

> **Inner function lives as long as something is referencing it — NOT till entire execution**

Proof — Without Variable (Not Referenced)

```
func main() {
    counter()()  // called directly — not stored anywhere

    // inner function immediately dies here
    // count immediately deleted here
    // NOT waiting for main() to end
}
```

Timeline:
counter() called → count created → inner function created
→ inner function called once → inner function NOT stored anywhere
→ IMMEDIATELY garbage collected
→ count IMMEDIATELY deleted
→ main() continues...
→ main() ends

Inner function died **immediately** — not at end of execution.

Proof — With Variable (Referenced)

```
func main() {
    increment := counter()  // inner function referenced by increment

    increment()  // 1
    increment()  // 2
    increment()  // 3

} // HERE — increment goes out of scope
  // NOW inner function dies
  // NOW count gets deleted
```

Timeline:
counter() called → count created → inner function created
→ stored in increment → increment references it
→ increment() called 3 times → count persists
→ main() ends → increment gone → nothing references inner function
→ NOW garbage collector deletes inner function AND count

Even Clearer Proof — Variable Goes Out of Scope Early

```
func main() {

    {  // inner block
       increment := counter()
       increment()  // 1
       increment()  // 2
    }  // HERE — increment goes out of scope
      // inner function dies HERE
      // count deleted HERE
      // main() is still running!

    // main() continues here
    // but inner function and count are already gone

    fmt.Println("main still running...")

}  // main ends here — much later
```

See — inner function died **in the middle of main()** — not at end of execution.

The Garbage Collector Rule

Variable/Function is deleted when:
→ Nothing is referencing it anymore

NOT when:
→ Entire program ends
→ main() ends
→ Any specific time

Summary Table

Situation	Inner function lives till
Not stored in variable	Immediately after that line
Stored in variable in main()	main() ends

Stored in variable in a block `{}`	Block ends
Stored in global variable	Entire program ends
Passed as argument	That function call ends

One Line Answer

Inner function lives **exactly as long as something references it** — could be 1 millisecond or entire program — depends purely on references, not execution time.